

Manipulação *de arquivos II*

Além do conteúdo, programas precisam lidar com **caminhos e pastas**: montar o nome de um arquivo, saber se ele existe, listar o que tem numa pasta, criar, renomear, copiar e apagar. O módulo `pathlib` faz isso com objetos `Path`, e o `shutil` cuida das cópias e das pastas inteiras. Nesta aula: as partes de um caminho, ler e gravar direto pelo `Path`, listar com `iterdir`, `glob` e `rglob`, criar e renomear, e as operações do `shutil`.

`Path` · operador `/`

`name` · `stem` · `suffix` · `parent`

`read_text` · `write_text`

`iterdir` · `glob` · `rglob`

`mkdir` · `rename` · `unlink`

`shutil`

Formação Claude Code 2026 · Baseado em docs.python.org (pathlib; shutil; os.stat_result), consultados em outubro de 2026 · Exemplos testados com Python 3.14 (Linux) em `exemplos/` · Saídas reais

Montando e *desmontando* caminhos

`caminhos.py`

```
from pathlib import Path

arquivo = Path("relatorios") / "2026" / "vendas_outubro.csv"
print(arquivo)           # o / junta partes do caminho
print(arquivo.name)
print(arquivo.stem, "|", arquivo.suffix)
print(arquivo.parent)
print(arquivo.with_suffix(".xlsx"))
print(arquivo.parts)
```

SAÍDA

```
relatorios/2026/vendas_outubro.csv
vendas_outubro.csv
vendas_outubro | .csv
relatorios/2026
relatorios/2026/vendas_outubro.xlsx
('relatorios', '2026', 'vendas_outubro.csv')
```

- ✓ O `/` entre `Path` e textos junta as partes com o separador certo do sistema
- ✓ `name` : nome completo; `stem` : sem extensão; `suffix` : a extensão
- ✓ `parent` : a pasta; `with_suffix` : o mesmo caminho com outra extensão
- ✓ Nada disso acessa o disco: é só manipulação do caminho

No Windows, o mesmo código mostra

`relatorios\2026\vendas_outubro.csv` . Por isso, monte caminhos com `/` do `Path` , nunca concatenando texto.

Atalhos do Path *e perguntas ao disco*

ler_gravar.py

```
from pathlib import Path

nota = Path("lembrete.txt")
nota.write_text("comprar café\n", encoding="utf-8")
print(nota.read_text(encoding="utf-8"), end="")
print(nota.exists(), nota.is_file(), nota.is_dir())
print(nota.stat().st_size, "bytes")
print(Path("sumiu.txt").exists())
```

SAÍDA

```
comprar café
True True False
14 bytes
False
```

- ✓ `write_text` e `read_text` abrem, gravam ou leem e fecham numa linha só
- ✓ `exists()`, `is_file()` e `is_dir()` perguntam ao disco
- ✓ `stat().st_size` dá o tamanho em bytes ("café" tem acento: 14, não 13)

Para arquivos pequenos, os atalhos bastam; para ler aos poucos, continue com `open` e `for` (aula 16).

Preparando uma *área para os exemplos*

preparar.py

```
"""Cria a pasta area_teste/ usada nos exemplos desta aula."""
import shutil
from pathlib import Path

def preparar():
    raiz = Path("area_teste")
    if raiz.exists():
        shutil.rmtree(raiz)          # recomeça do zero
    (raiz / "fotos").mkdir(parents=True)
    (raiz / "docs" / "2026").mkdir(parents=True)
    for nome in ["praia.jpg", "festa.jpg", "logo.png"]:
        (raiz / "fotos" / nome).write_bytes(b"\x00" * 10)
    (raiz / "docs" / "notas.txt").write_text("8\n9\n", "utf-8")
    plano = raiz / "docs" / "2026" / "plano.md"
    plano.write_text("# Plano", "utf-8")
    return raiz
```

AREA_TESTE/
raiz



FOTOS/
3 imagens



DOCS/
notas.txt ·
2026/plano.md

- ✓ `mkdir(parents=True)` cria também as pastas do meio
- ✓ `write_bytes` cria arquivos binários de mentira, só para existirem
- ✓ `shutil.rmtree` apaga a pasta antiga: cada exemplo começa do mesmo estado
- ✓ Os próximos exemplos importam `preparar()` e mexem nessa pasta

Teste operações de apagar e mover numa pasta só para isso, nunca nos seus arquivos de verdade.

iterdir, glob e rglob

listar.py

```
from preparar import preparar

raiz = preparar()

print(sorted(p.name for p in raiz.iterdir()))      # 1 nível
print(sorted(p.name for p in raiz.glob("fotos/*.jpg")))
for p in sorted(raiz.rglob("*..*")):              # todos os níveis
    print(p.relative_to(raiz))
```

SAÍDA

```
['docs', 'fotos']
['festa.jpg', 'praia.jpg']
docs/2026/plano.md
docs/notas.txt
fotos/festa.jpg
fotos/logo.png
fotos/praia.jpg
```

MÉTODO

DEVOLVE

`iterdir()`

o que está direto na pasta

`glob("fotos/*.jpg")`

o que casa com o padrão

`rglob("*..*")`

o padrão em todas as subpastas

- ✓ Os três devolvem **iteradores** de `Path`
- ✓ A ordem não é garantida: use `sorted` quando ela importar
- ✓ `relative_to(raiz)` mostra o caminho a partir da pasta

Mexendo em *pastas e arquivos*

pastas.py

```
from pathlib import Path
from preparar import preparar

raiz = preparar()
destino = raiz / "backup" / "outubro"
destino.mkdir(parents=True, exist_ok=True) # cria tudo
destino.mkdir(parents=True, exist_ok=True) # não reclama
print(destino.is_dir())

antigo = raiz / "docs" / "notas.txt"
novo = antigo.rename(raiz / "docs" / "notas_2026.txt")
print(novo.name, antigo.exists())

(raiz / "fotos" / "logo.png").unlink() # apaga
print(sorted(p.name for p in (raiz / "fotos").iterdir()))
```

SAÍDA

```
True
notas_2026.txt False
['festa.jpg', 'praia.jpg']
```

- ✓ `mkdir(parents=True, exist_ok=True)` cria o caminho todo e não dá erro se já existir
- ✓ `rename` renomeia ou move, e devolve o novo caminho
- ✓ `unlink()` apaga um arquivo; `rmdir()`, uma pasta vazia

Apagar pelo Python não manda para a lixeira: o arquivo some de vez.

Copiar, mover *e apagar pastas inteiras*

`copiar.py`

```
import shutil
from preparar import preparar

raiz = preparar()
copia = shutil.copy(raiz / "docs" / "notas.txt", raiz / "fotos")
print(copia)

shutil.copytree(raiz / "fotos", raiz / "fotos_bkp") # pasta
shutil.move(raiz / "docs" / "2026", raiz / "morto") # move
shutil.rmtree(raiz / "fotos_bkp") # apaga tudo
print(sorted(p.name for p in raiz.iterdir()))
```

SAÍDA

```
area_teste/fotos/notas.txt
['docs', 'fotos', 'morto']
```

FUNÇÃO

FAZ

`shutil.copy(origem, destino)`

copia um arquivo (destino pode ser uma pasta)

`shutil.copytree(a, b)`

copia uma pasta com tudo dentro

`shutil.move(a, b)`

move arquivo ou pasta

`shutil.rmtree(pasta)`

apaga a pasta e todo o conteúdo

`rmtree` não pergunta nada. Confira o caminho antes de chamar.

Organizar *os downloads*

Enunciado: mova cada arquivo de uma pasta bagunçada para uma subpasta com o nome da extensão (jpg, pdf, txt...), ignorando maiúsculas.

Uma resposta possível

```
from pathlib import Path
from preparar import preparar

raiz = preparar()
baguncada = raiz / "downloads"
baguncada.mkdir()
for nome in ["foto.jpg", "nota.pdf", "selfie.JPG", "doc.txt"]:
    (baguncada / nome).write_text("", encoding="utf-8")

for arquivo in list(baguncada.iterdir()):  # lista antes
    tipo = arquivo.suffix.lower().lstrip(".") or "sem_tipo"
    pasta = baguncada / tipo
    pasta.mkdir(exist_ok=True)
    arquivo.rename(pasta / arquivo.name)

for pasta in sorted(baguncada.iterdir()):
    print(pasta.name, sorted(p.name for p in pasta.iterdir()))
```

SAÍDA

```
jpg ['foto.jpg', 'selfie.JPG']
pdf ['nota.pdf']
txt ['doc.txt']
```

- ✓ `list(...)` guarda a lista antes de mover: não é seguro mudar uma pasta enquanto ela é percorrida
- ✓ `suffix.lower()` junta `.JPG` e `.jpg`
- ✓ Desafio: se já existir um arquivo com o mesmo nome no destino, acrescente um número

Teste o que *ficou*

1. `Path("a/relatorio.pdf").stem` vale:

A) "relatorio.pdf"

B) "relatorio"

C) ".pdf"

D) "a"

B.

2. Para listar todos os `.txt` em todas as subpastas:

A) `iterdir()`

B) `glob("*.txt")`

C) `rglob("*.txt")`

D) `listdir()`

C.

3. `mkdir` sem erro se a pasta já existir:

A) `mkdir()`

B) `mkdir(exist_ok=True)`

C) `mkdir(force=True)`

D) `makedirs()`

B.

4. Qual apaga uma pasta com tudo dentro?

A) `Path.unlink()`

B) `Path.rmdir()`

C) `shutil.rmtree()`

D) `shutil.copy()`

C.

Caminhos e pastas

sem dor de cabeça

Monte caminhos com `Path` e `/`, use `name`, `stem`, `suffix` e `parent` para desmontá-los, e `read_text` / `write_text` para arquivos pequenos. Liste com `iterdir`, `glob` e `rglob`; crie com `mkdir(parents=True, exist_ok=True)`; renomeie e apague com cuidado, e use o `shutil` para copiar e mover pastas inteiras.

`Path` /

`stem` · `suffix`

`glob` · `rglob`

`mkdir` · `rename`

`shutil`

Próxima aula → 18 · Manipulação de Arquivos III

Fontes: docs.python.org/3/library/pathlib.html; docs.python.org/3/library/shutil.html; docs.python.org/3/library/os.html#os.stat_result, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4 no Linux.